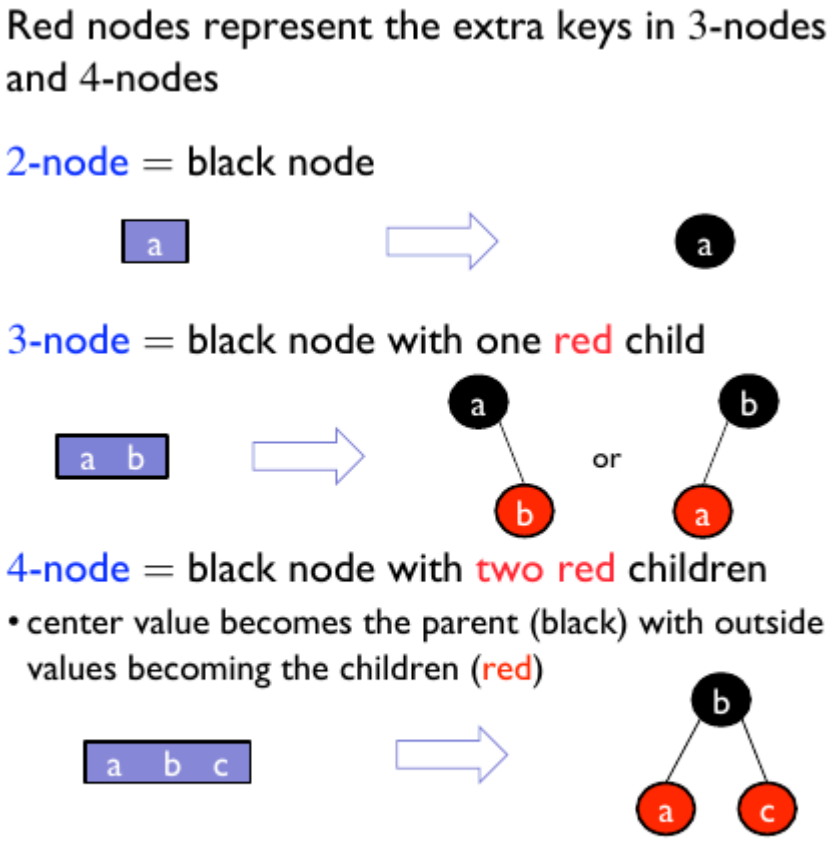
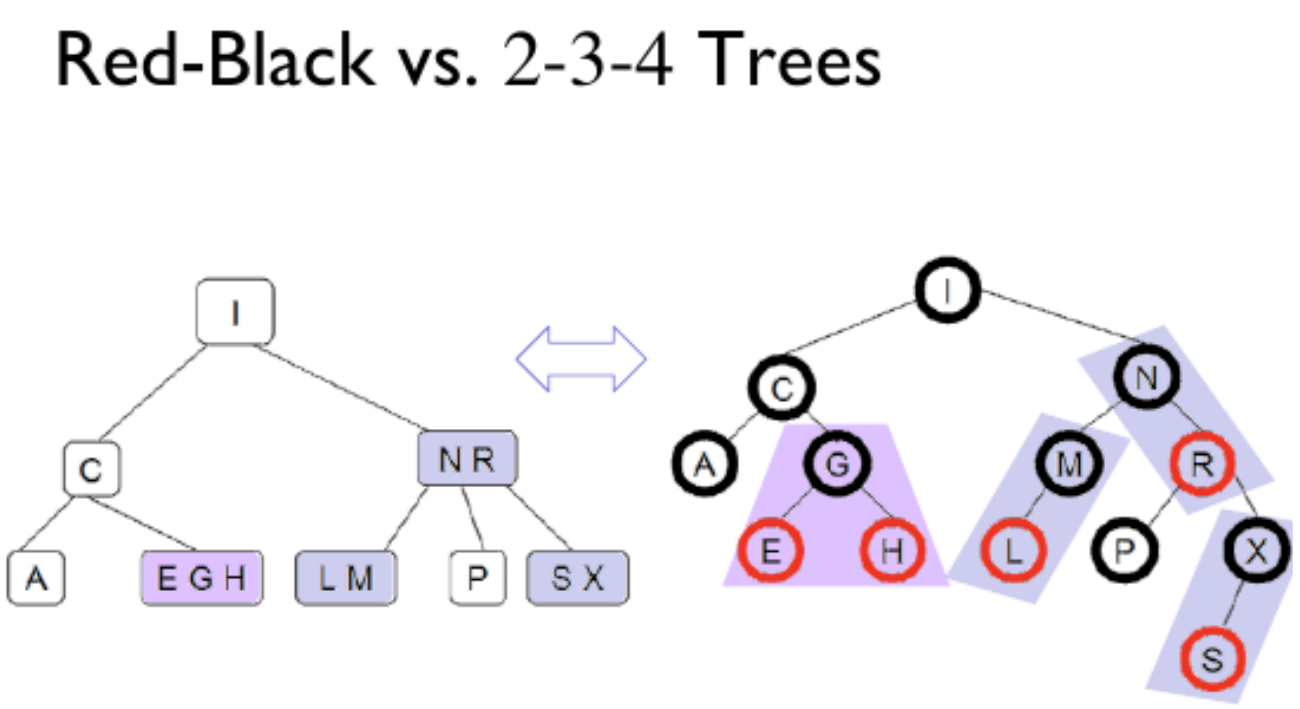
Red black trees are similar to 2-3-4 Tree with minimal changes to the original Binary Search Tree hence making Red black trees similar to a conventional `bst` and making it self-balancing similar to a `2-3-4 tree`.

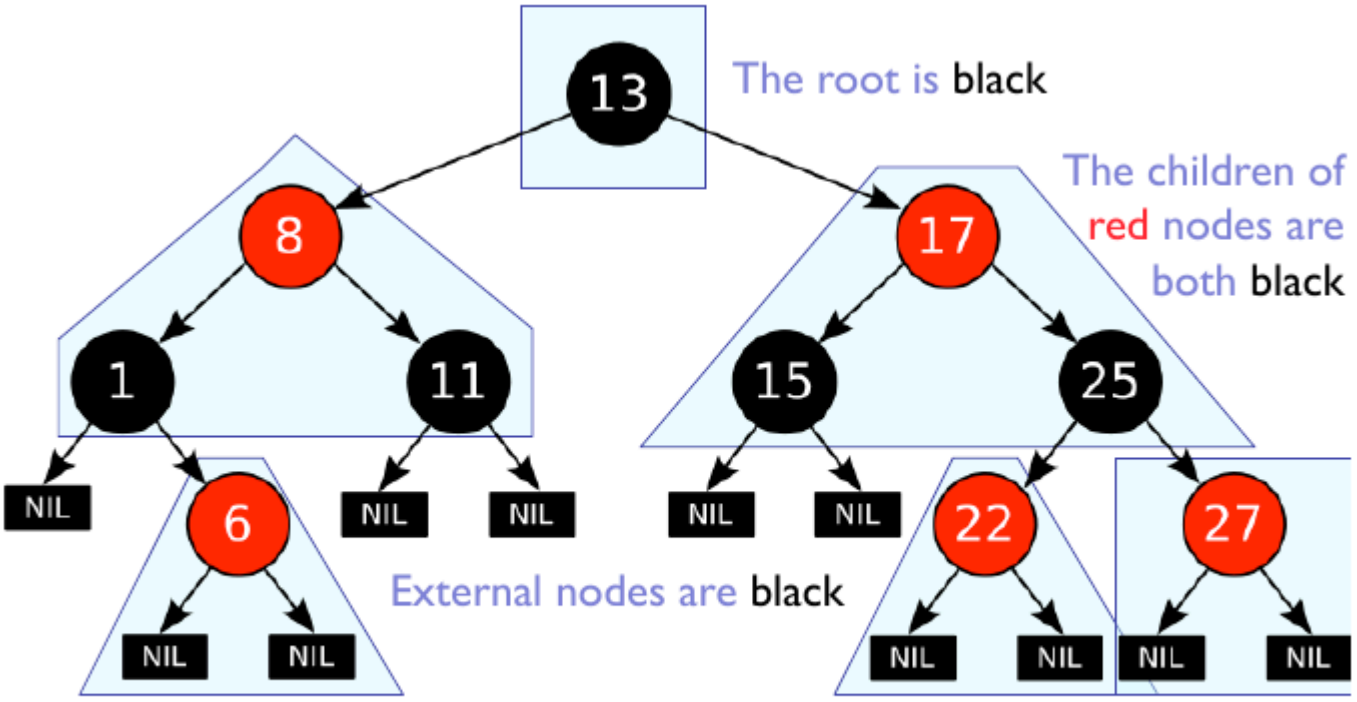


Red-black trees are not unique, but the corresponding 2-3-4 tree is unique [1]



Properties of a Red Black Tree

Looking at the above analogy to a `2-3-4 tree` we can immediately derive some properties:

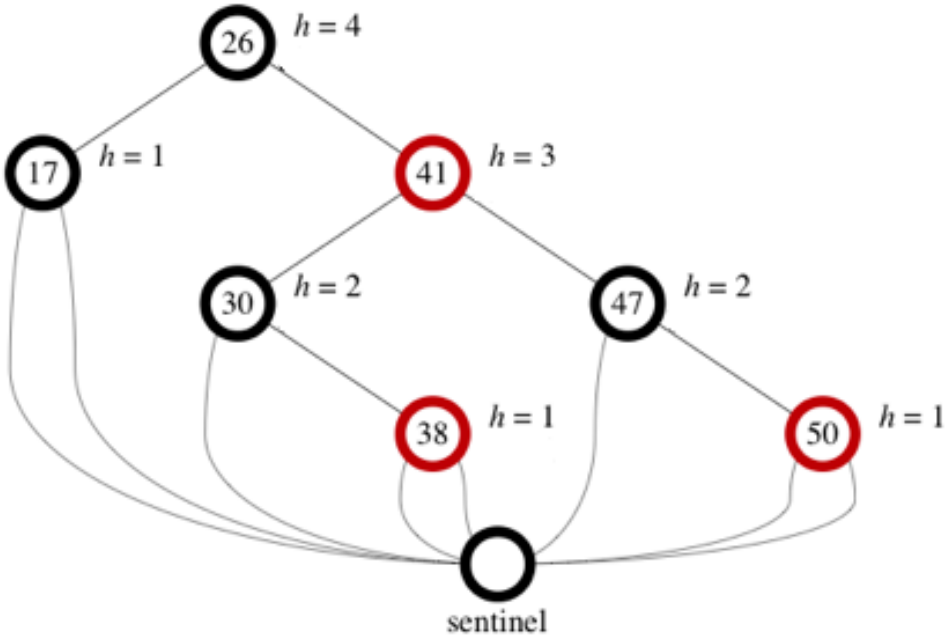


Every node is colored either red or black

Notice that in a red black tree `external nodes` / `leaf nodes` are considered to be `nil`. So in the above example the following nodes which are usually considered leaf are instead internal nodes: 6, 11, 15, 22, 27

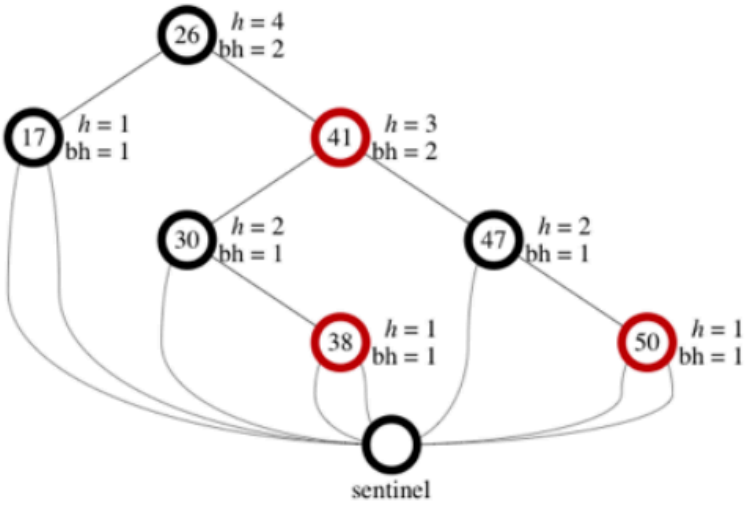
So the height of a `red black tree` get's counted from the `nil` / `sentinel` node:

The **height of a node** is the number of edges in a longest path to a leaf.



red black tree have an additional attribute which is called `black-height`:

The **black-height** of node x, which we write as $\mathrm{bh}(x)$, is the number of black nodes (including the sentinel) on the path from x to a leaf, not counting x.



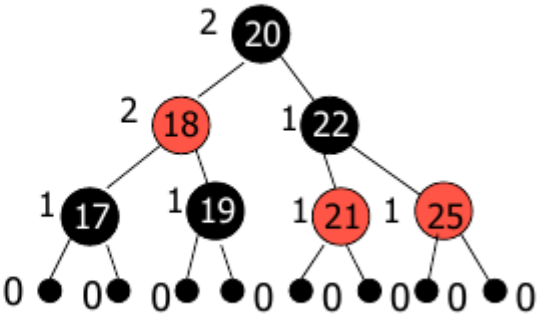
The `black height` leads us to another property of the red black tree:

- For each node, all paths from the node to `descendant leaves` contain the same number of black nodes. For example:

Every node has a black-height, $\mathrm{bh}(x)$

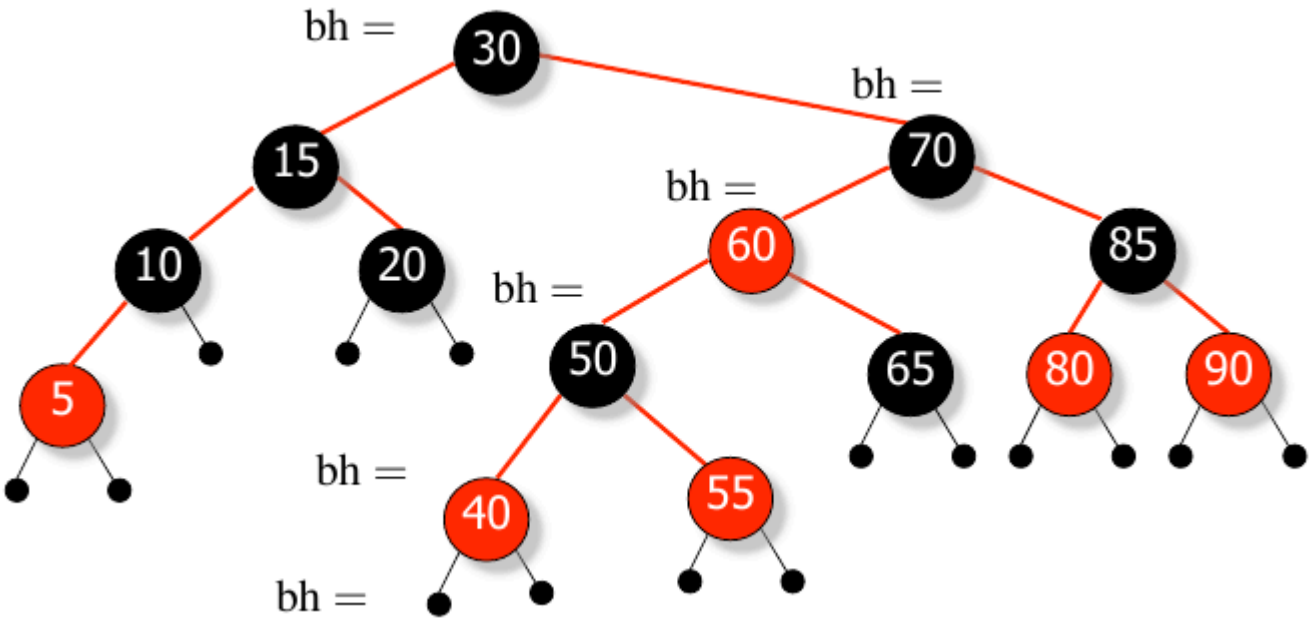
For all external nodes, $\mathrm{bh}(x) = 0$

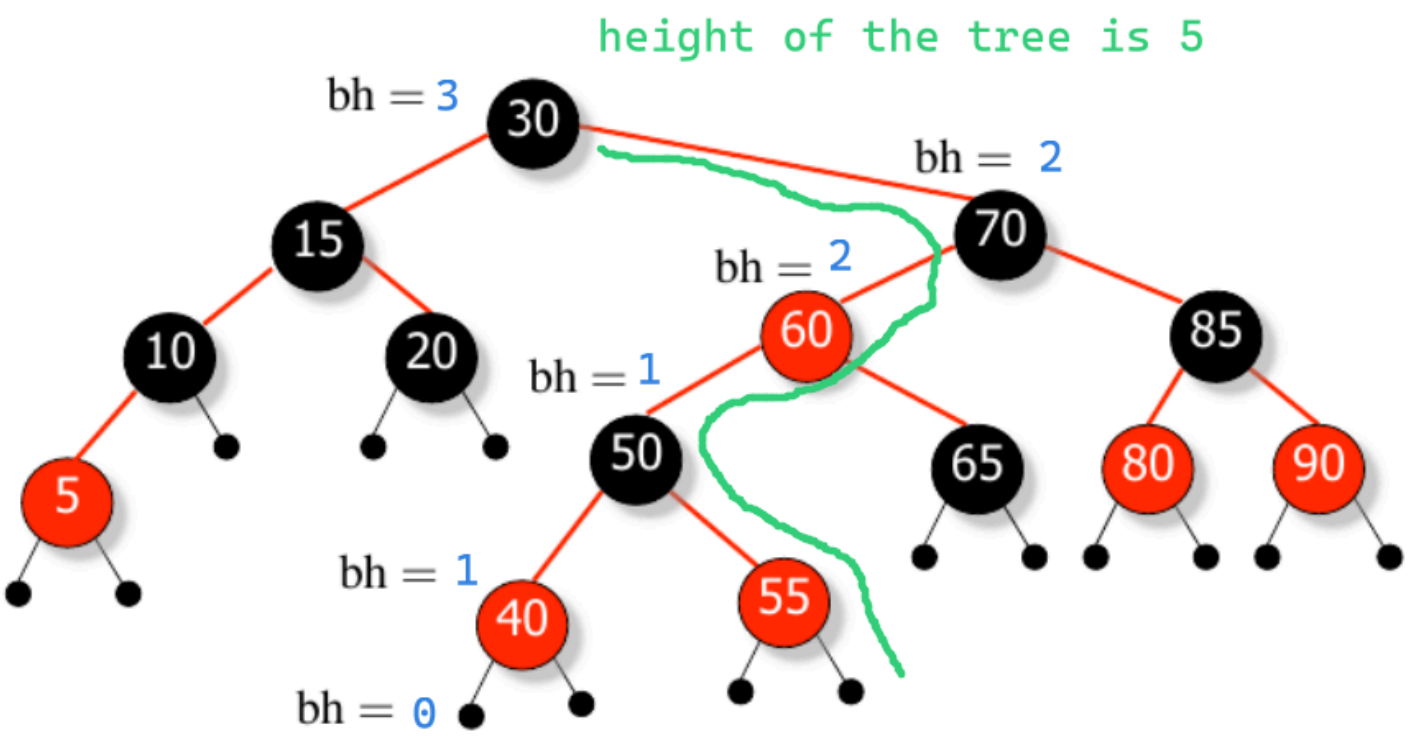
For root x, $\mathrm{bh}(x) = \mathrm{bh}(T)$



This property of the red black tree is analog to the of height increase property of the `2-3-4 tree` which makes the red black tree a self balancing binary search tree.

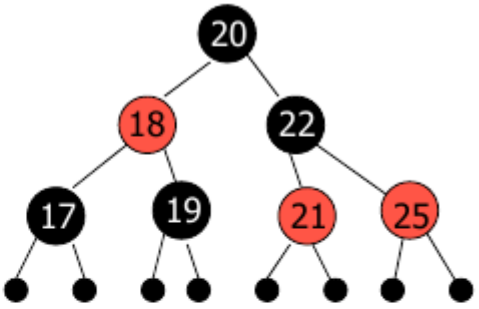
Fill the black height:





So in summary the properties of a red black tree are:

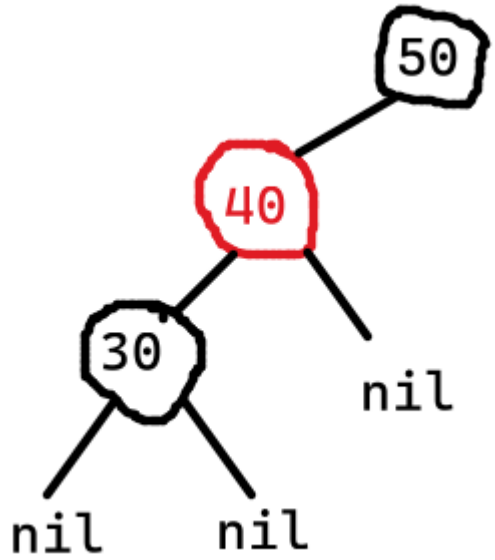
1. Every node is either red or black
2. The root is black [root rule]
3. External nodes (nulls) are black
4. If a node is red, then both its children are black [red rule]



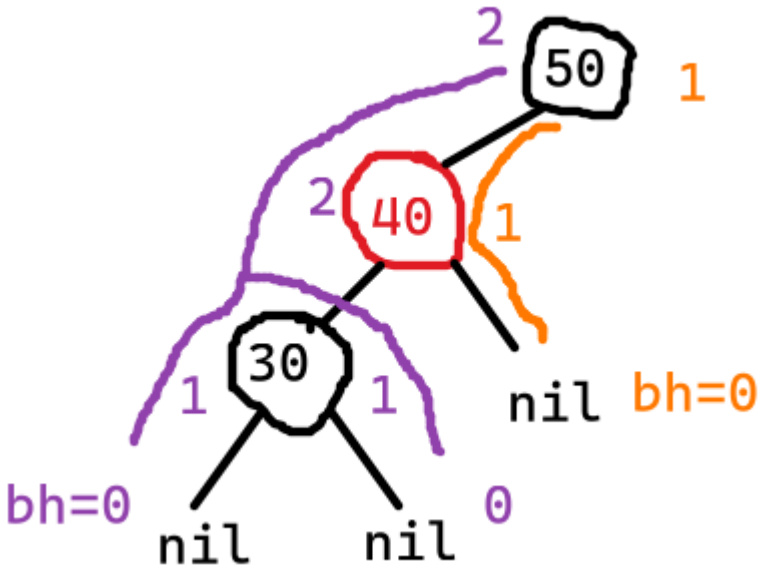
5. Every path from a node to a null must have the same number of black nodes (black height) [black-height rule]
   a. this is equivalent to a 2-3-4 tree being a perfect tree: all the leaf nodes of the 2-3-4 tree are at the same level (black-height=1)
   b. a black node corresponds to a level change in the corresponding 2-3-4 tree

1. Why does a red node must have two black children? Why can't a red node have a single black child?[2]

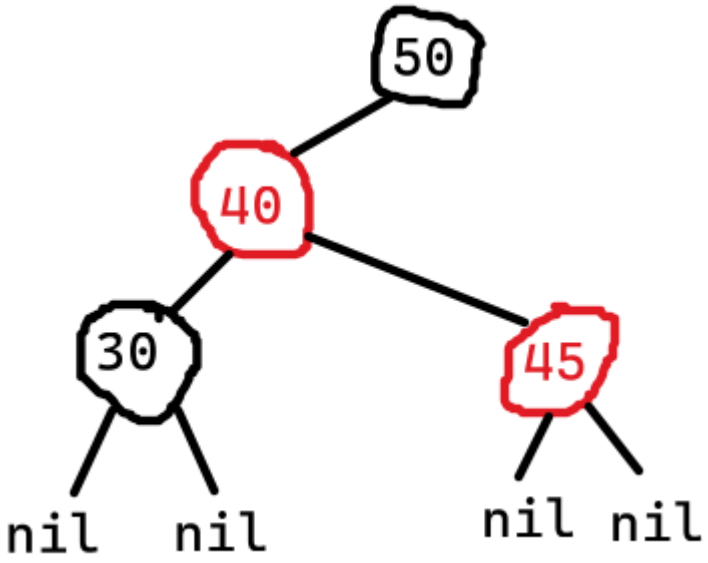
Because if we were to have something like:

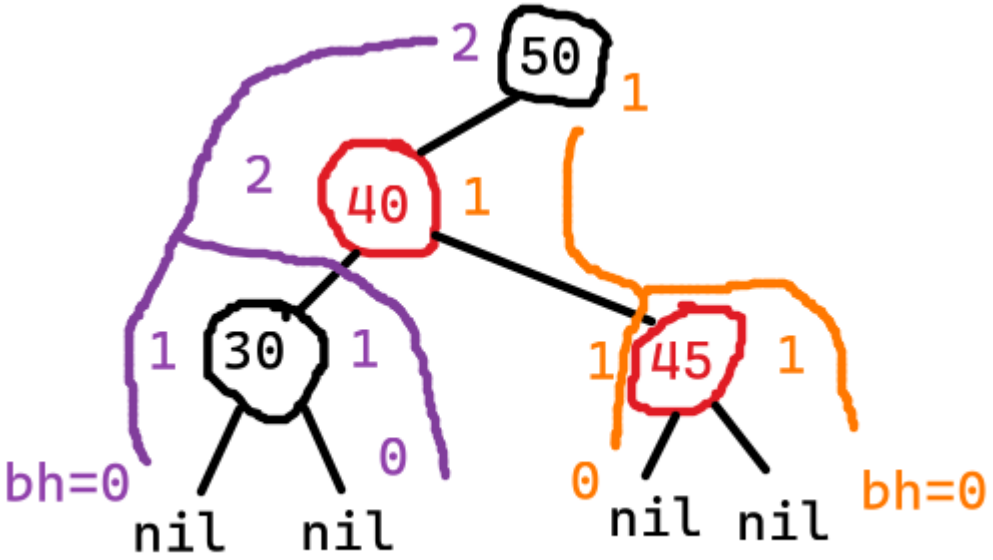


Then the `black height` would be different which would violate the `self balancing property` of the red black tree:



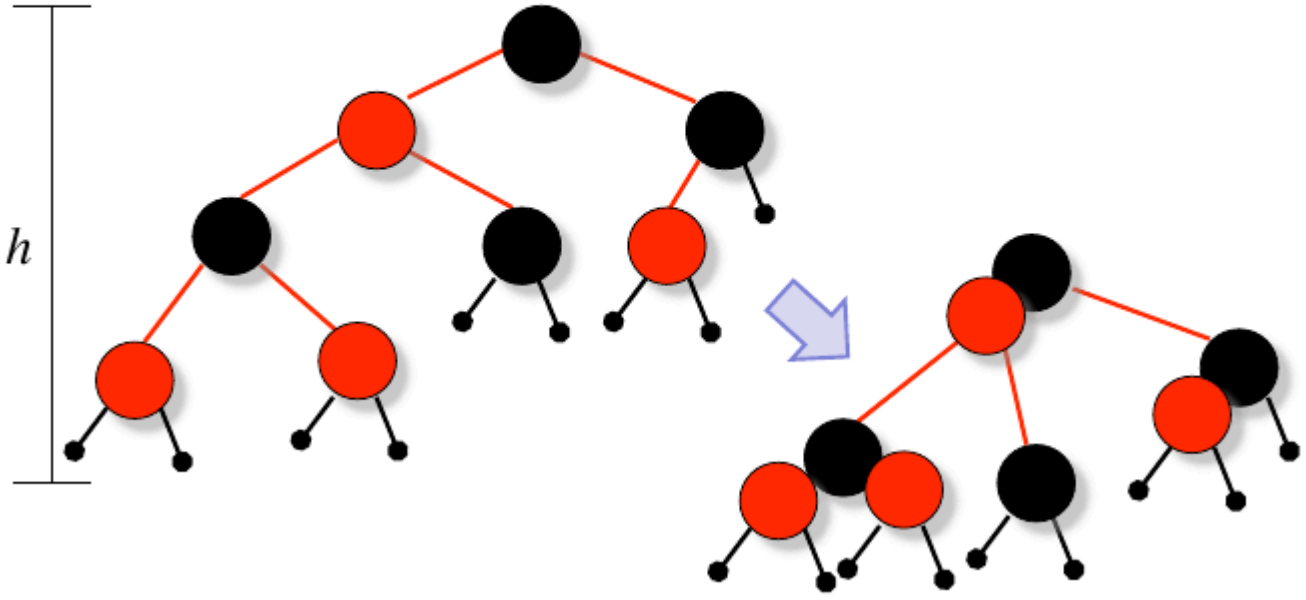
If a node N has exactly one child, the child must be red. If the child were black, its leaves would sit at a different black depth than N's null node. If a node N has two children then they can either be both red or both black but not alternate. Because alternative coloring would again violate the `black height` rule:





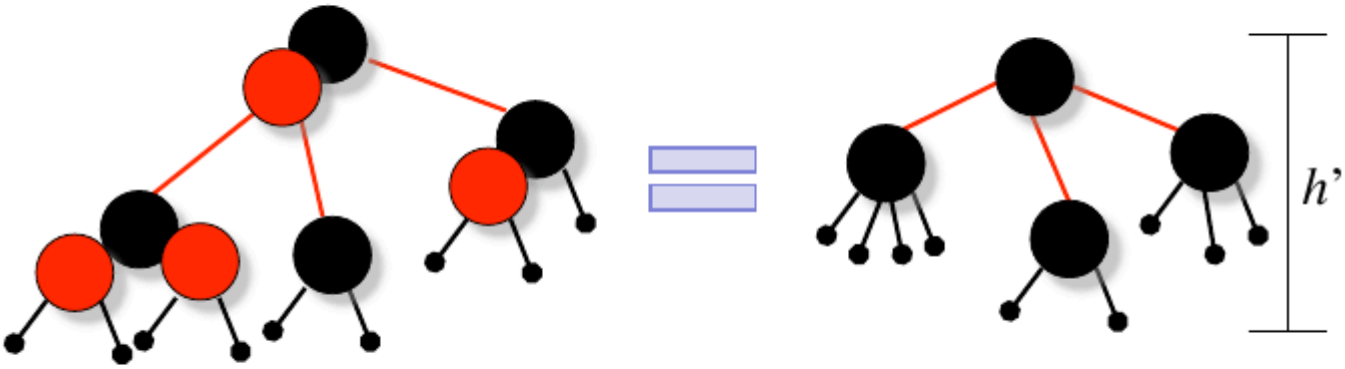
Conversion to equivalent `2-3-4 tree` representation

Start with a red black tree with height h
(note: *height* here includes the external nodes)
Merge all red nodes into their black parents



Nodes in resulting tree have degrees between 2 and 4

All external nodes are at the same level



It's the equivalent 2-3-4 tree to the red-black tree!

Height of the resulting tree is $h' \geq h/2$

Insertion

First we do insertion similar to the Binary Search Tree and then check whether we just inserted into `2 node`, `3 node`, `4 node`. After the determination of the type of node we apply rotation and recoloring accordingly.

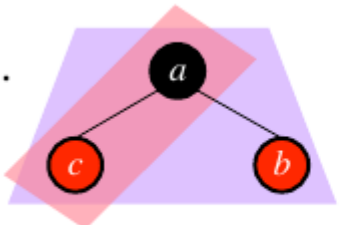
Insertion into a `2 node`

Insertion into a `2 node` is simple. You just insert according to the `bst` property.

Insertion into a `3 node`

Insertion into a `3 node` has two cases:

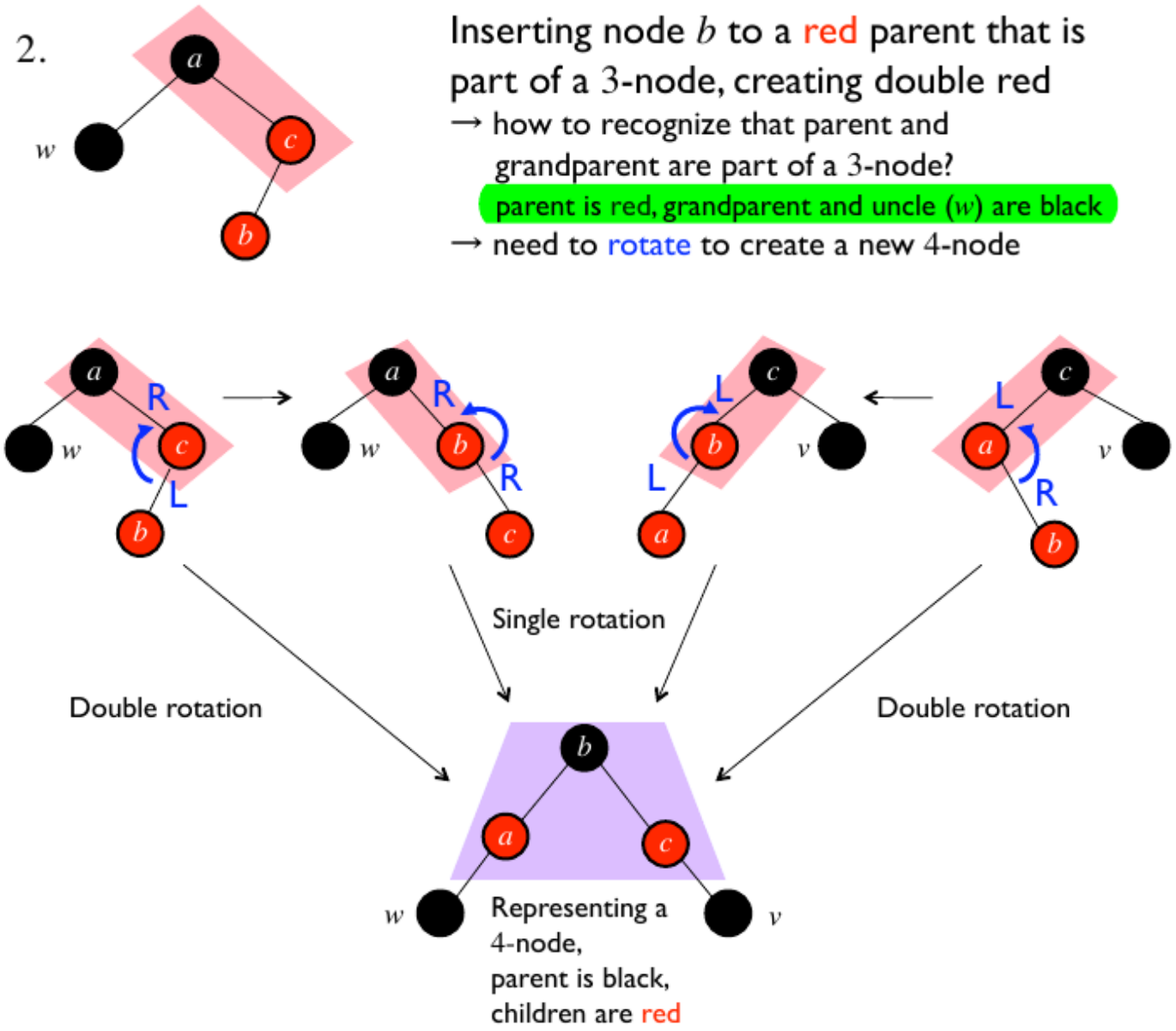
1. If the parent is black then the case is as simple as the `2 node` case:

1. 

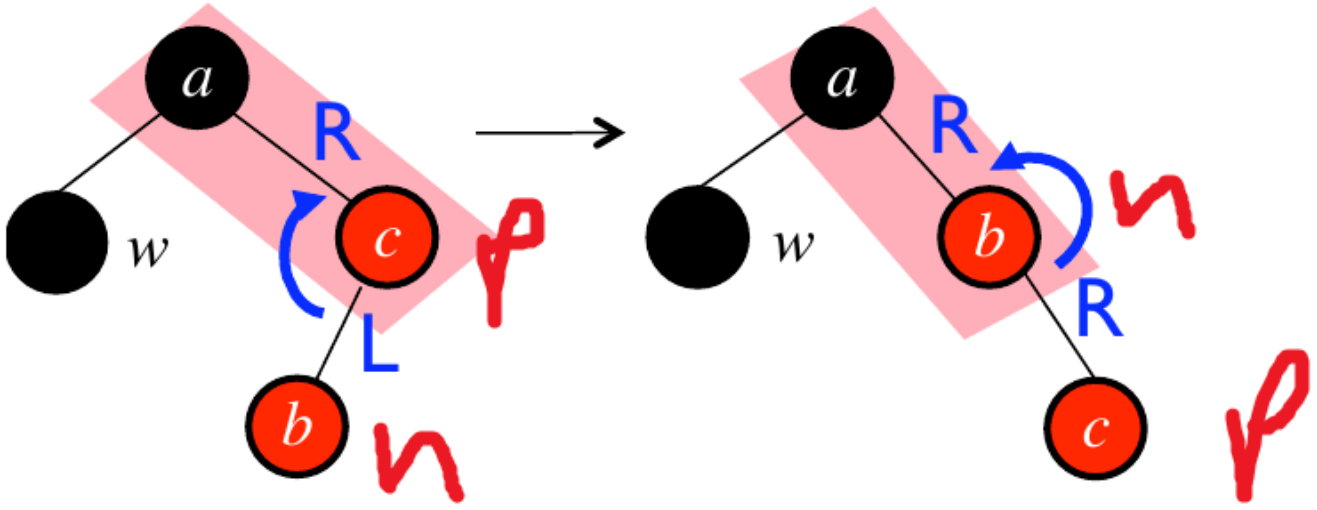
Inserting node b to a black parent that is part of a 3-node, creating a 4-node, done

⇒ inserting a new node to a black parent is always simple

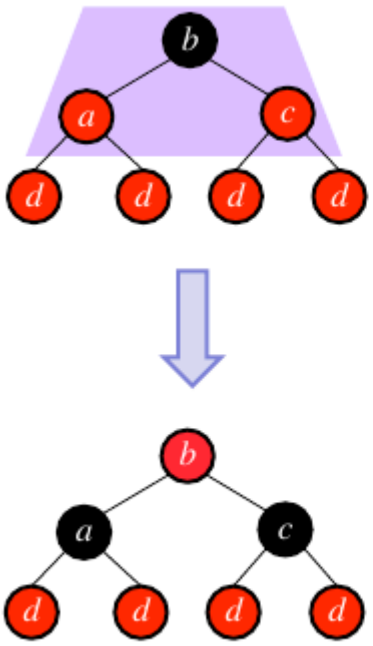
2. If the parent is red then we require rotation and recoloring:



The `RL` case reduces to `RR` case. While the `LR` case reduces to `LL` case. No matter the case the grandparent gets changed to red. The parent in the `RR` and `LL` cases get colored to black. While in the `RL` and the `LR` case we do an additional rotation and swap the parent pointer to move on to the base case of either `RR` or `LL` case. Example swapping of pointers on `RL` to `RR` case:



Insertion into a 4 node



Inserting node d causes double red, and d' s parent has red sibling w

→ parent, aunt, and grandparent are part of a 4-node

→ need to recolor, to split the 4-node and "promote" grandparent

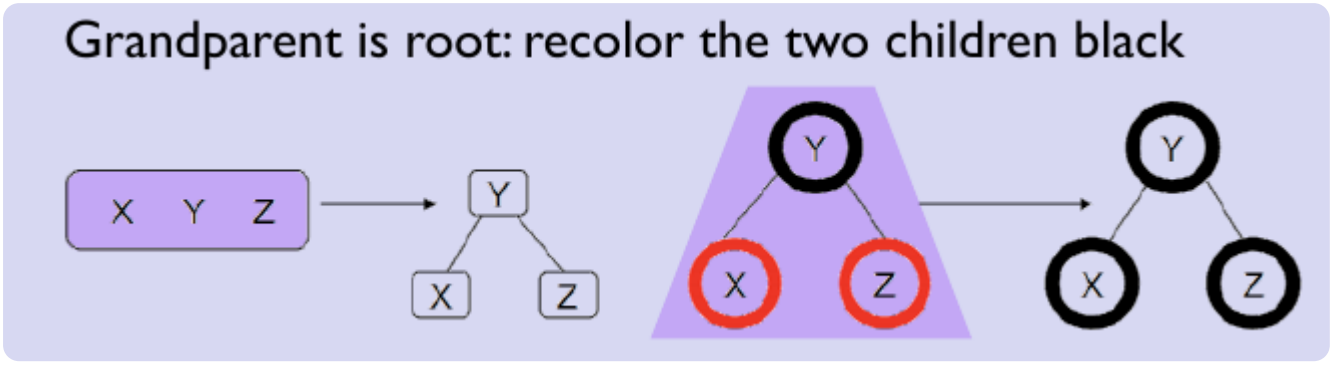
parent and aunt become black

grandparent becomes red

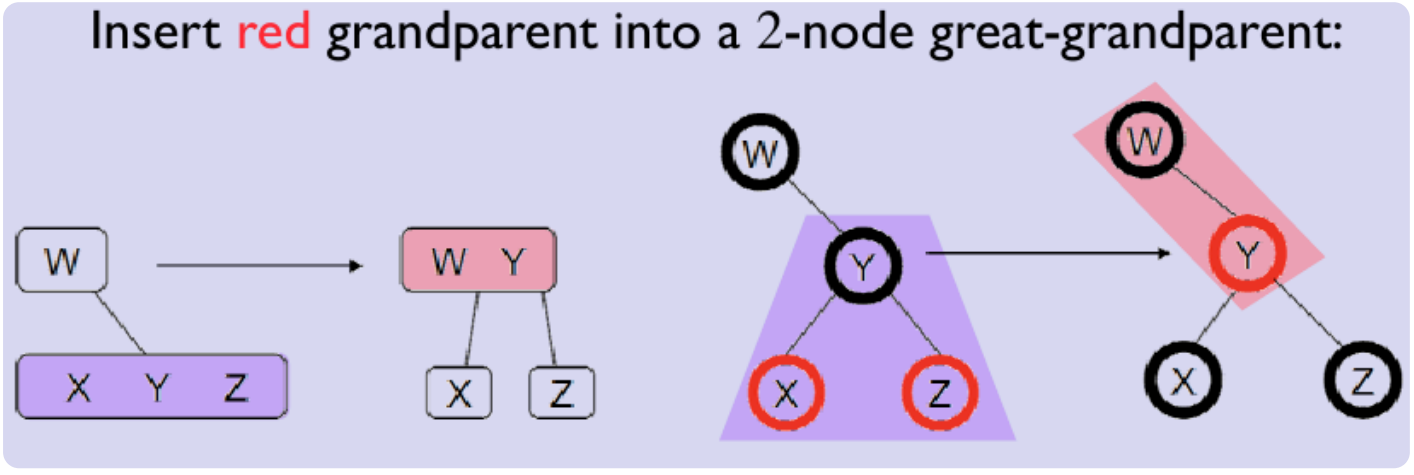
If grandparent is root, change it back to black

Otherwise, insert grandparent to great-grandparent, applying the same insertion rules as before depending on whether great-grandparent is a 2-node, 3-node, or 4-node

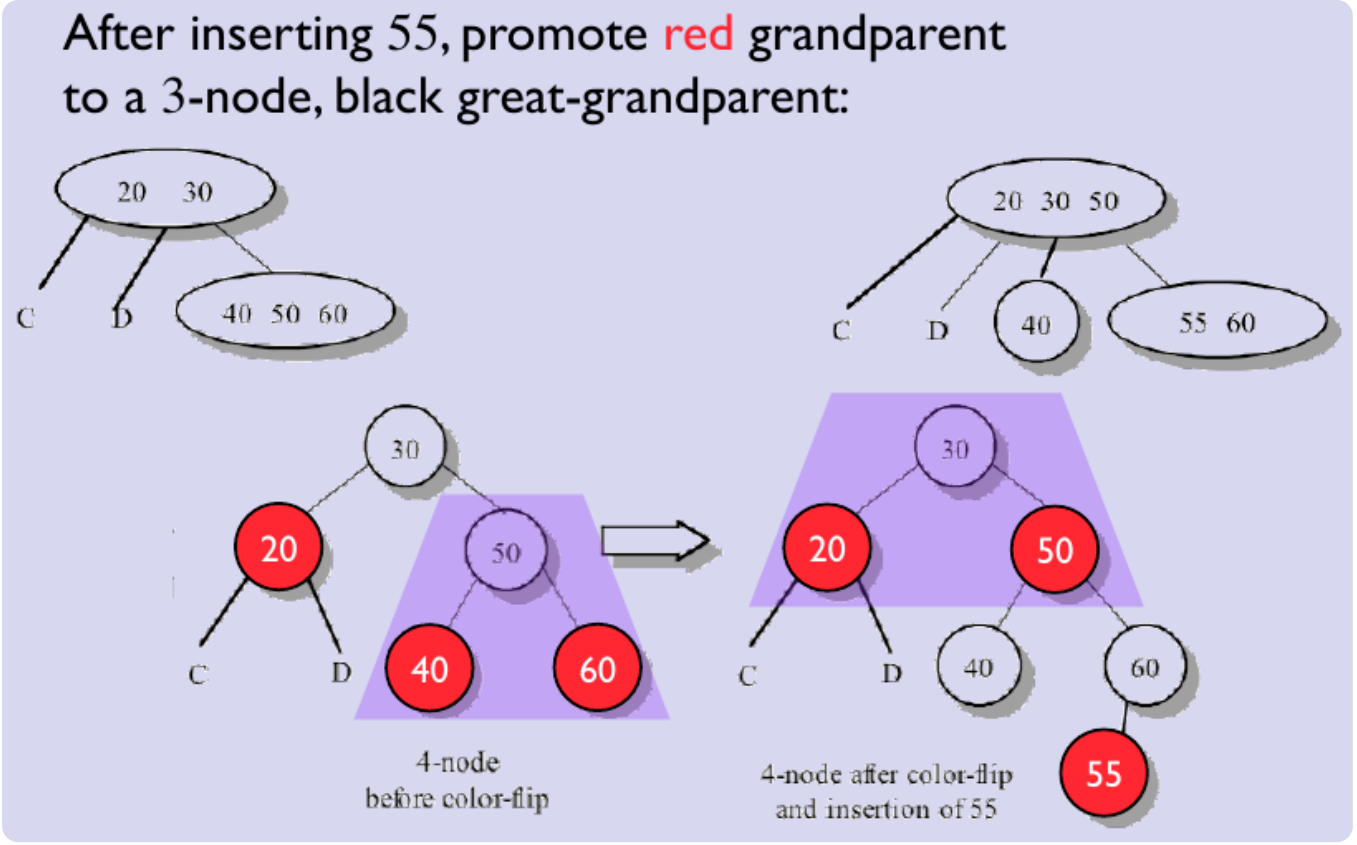
1. The most simple case is if the grandparent is the root of the tree:



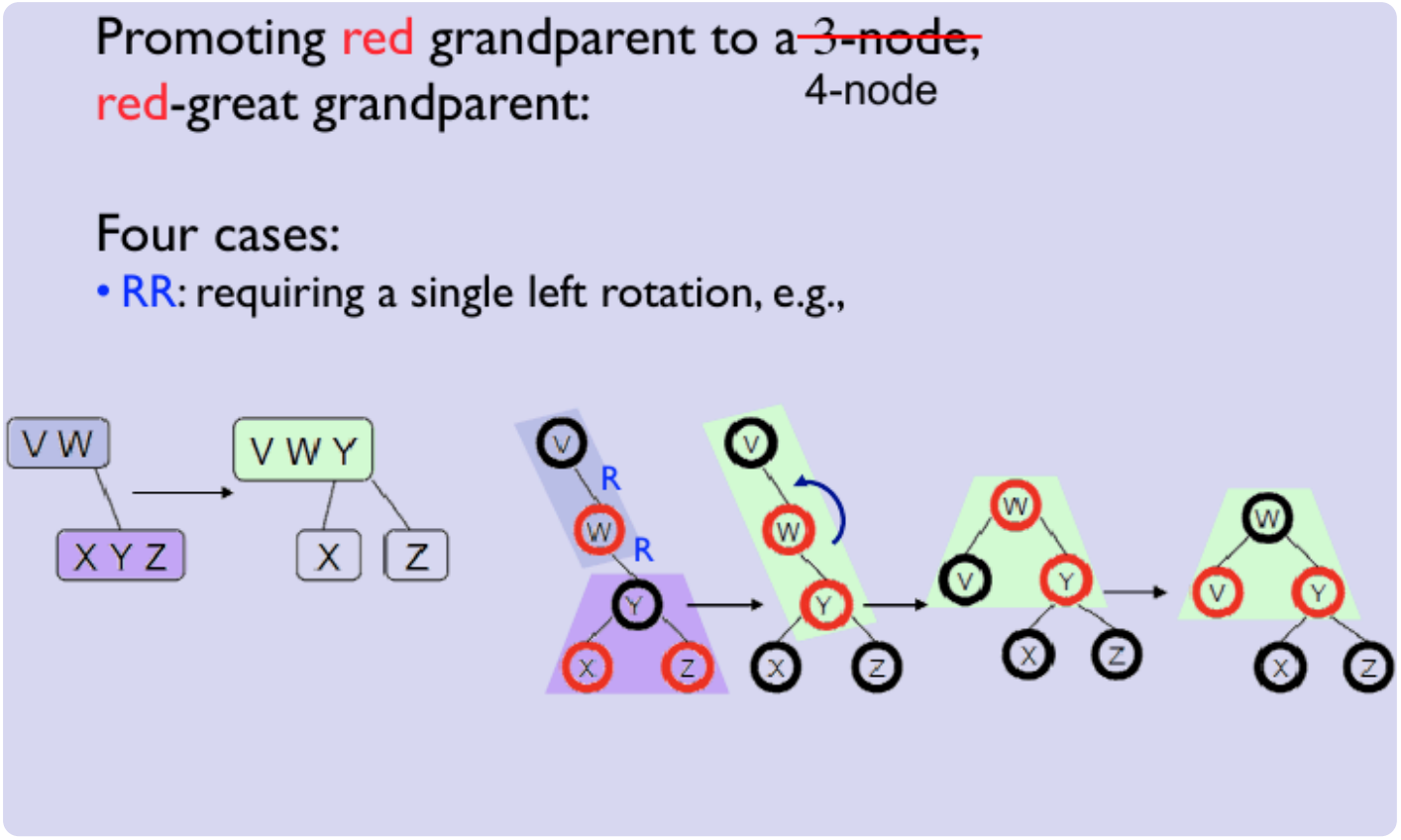
2. Another simple case is if the grandparent has a 2 node parent, i.e., great grandparent is 2 node :

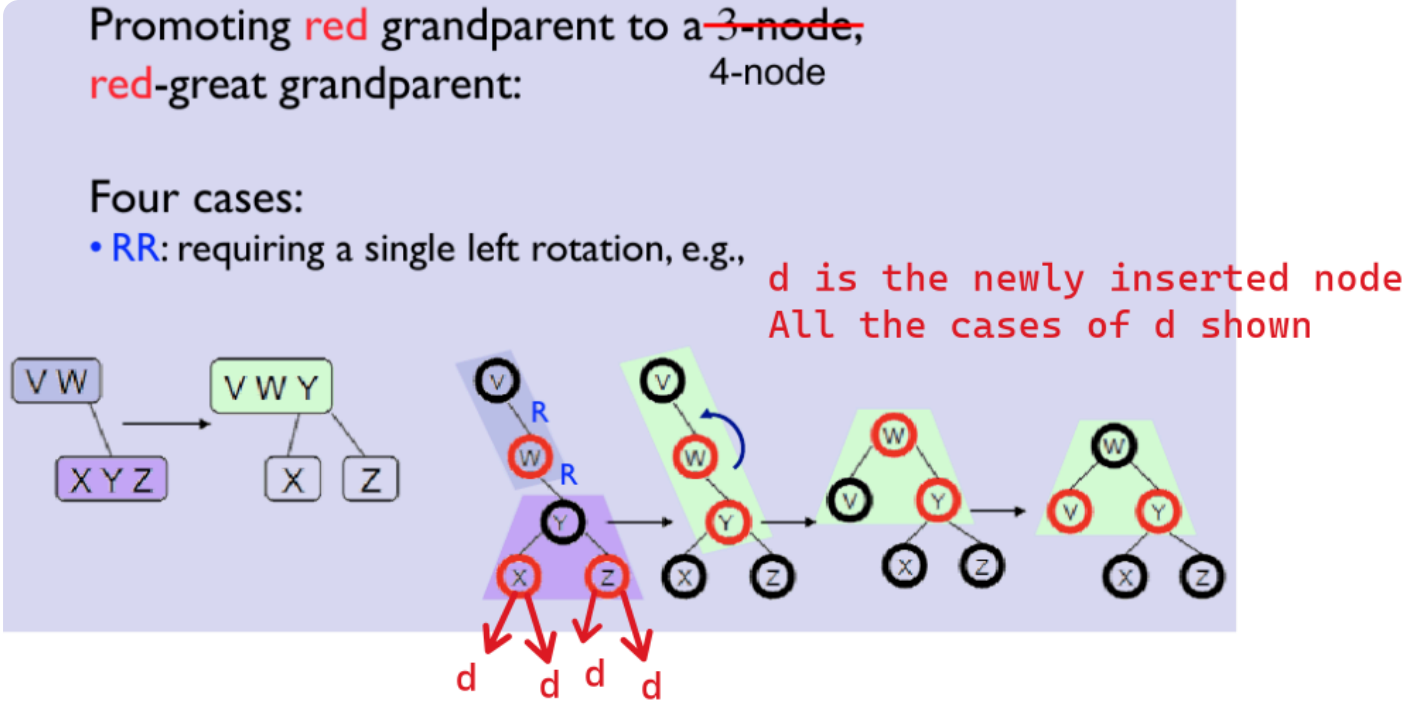


- This same case works if the great grandparent is a 3 node and we are opposite to the red child of the great grandparent:



3. If the great grandparent is part of a `3 node` but we are on the red side:





We first recolor then call then do the same procedure as `we did for a 3 node` from the point of view of great parent of d.

4. If the great grandparent is part of a `4 node`. Then we flip colors and recursively look for the 3 cases discussed above.

Deletion

Observations:

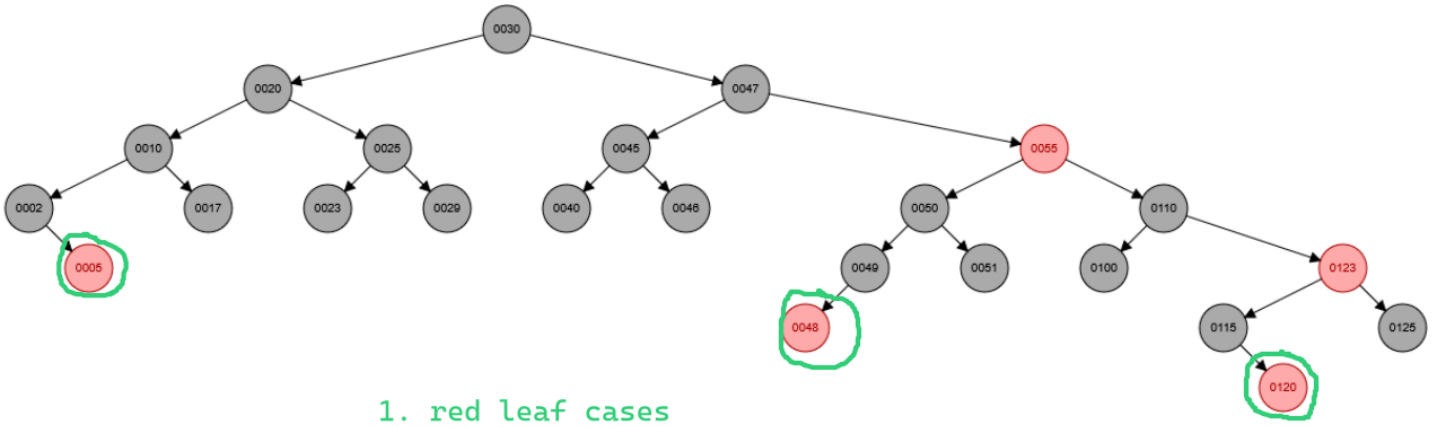
- if we delete a red node, tree is still a red-black tree
- a red node is either a leaf node or must have two children

Rules:

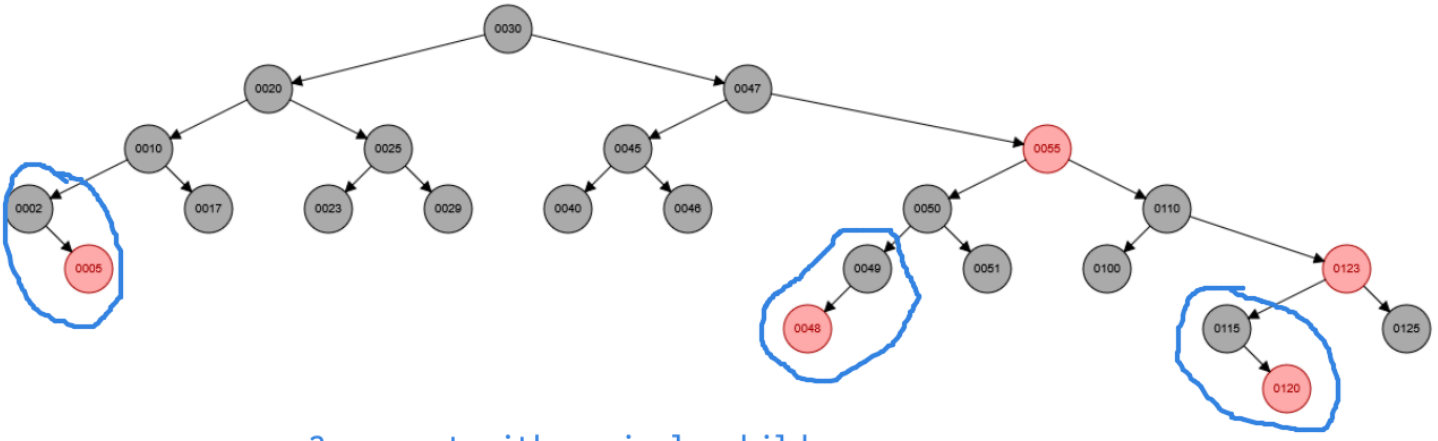
1. if node to be deleted is a red leaf, remove leaf, done
2. if it is a single-child parent, it must be black (why?); replace with its child (must be red) and recolor child black
3. if it has two internal node children, swap node to be deleted with its in-order successor
   - if in-order successor is red (must be a leaf, why?), remove leaf, done
   - if in-order successor is a single child parent, apply second rule

In both cases the resulting tree is a legit red-black tree (we haven't changed the number of black nodes in paths)

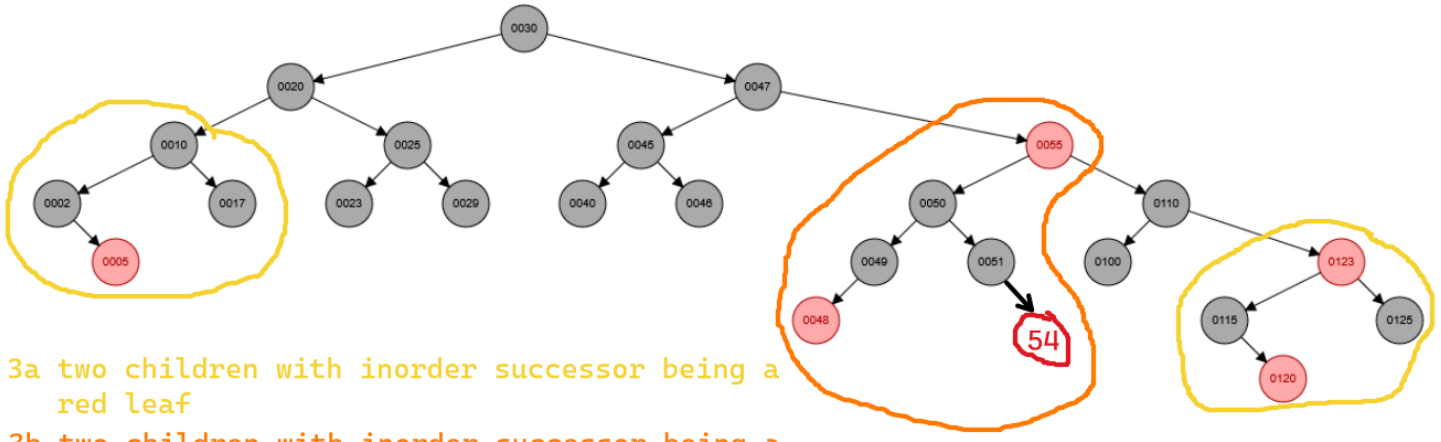
4. if in-order successor is a black leaf, or if the node to be deleted is itself a black leaf, things get complicated . . .



1. red leaf cases



2. parent with a single child case



3a two children with inorder successor being a red leaf

3b two children with inorder successor being a single parent with a red leaf

Note: We take the inorder successor here as the largest value of the left subtree

1. https://web.eecs.umich.edu/~sugih/courses/eecs281/f11/lectures/11-Redblack.pdf ↩
2. https://en.wikipedia.org/wiki/Red%E2%80%93black_tree#Properties ↩